

U3 · Aula 5 — Visual AI e Geração de Testes com LLM

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Visual AI:** pixel diff → perceptual → ML-based
- **Parte 2 — Geração com LLM:** API de LLM, few-shot, oracle problem
- **Parte 3 — Prática guiada:** pipeline de geração + visual diff (lab IA)

Depois desta aula: screencast — eu rodando o pipeline passo a passo.

Objetivos de aprendizagem

- **Comparar** os 3 paradigmas de visual testing e seus falsos positivos
- **Gerar** um teste Playwright via API de LLM a partir de user story
- **Explicar** o oracle problem — por que teste gerado pode passar errado
- **Medir** custo real (tokens, latência) de geração com LLM
- **Implementar** um visual diff com pixelmach e um gate de drift

Parte 1 — Visual AI

Pixel diff · perceptual diff · ML-based

Visual AI — três paradigmas

Paradigma	Como funciona	Falso positivo
Pixel diff	Compara pixel a pixel	Alto (anti-aliasing, sub-pixel, fontes)
Perceptual diff (SSIM / CIE ΔE)	Modelo da percepção humana	Médio
ML-based (Applitoools)	Modelo treinado em diffs anotados	Baixo

Em 2026, **ML-based é o padrão** em times sérios. Pixel diff ainda útil para assets específicos.

Applitools Eyes — integração Playwright

```
import { Eyes, Target } from '@appliTools/eyes-playwright';

test('tela de login visual', async ({ page }) => {
  const eyes = new Eyes();

  await eyes.open(page, 'MyApp', 'Login Screen');
  await page.goto('/login');

  // Check full page
  await eyes.check('Login form', Target.window().fully());

  // Check componente específico
  await eyes.check('Header', Target.region(page.getByRole('banner')));

  await eyes.close();
});
```

Trade-off: tolerância configurável — pode suprimir bug visual real. Auditar reports.

Modos de verificação AppliTools

Modo	O que detecta	Quando usar
Strict	Diferenças visuais exatas	Imagens, ícones críticos
Content	Mudança de texto	Copy, labels, mensagens de erro
Layout	Mudança de estrutura	Posicionamento, alinhamento
Ignore	Área excluída da comparação	Dados dinâmicos (timestamps, avatars)

```
await eyes.check('dashboard', Target.window()  
  .ignoreRegions(page.getByTestId('timestamp'))  
  .layout()  
);
```

Parte 2 — Geração de testes com LLM

API de LLM · few-shot · oracle problem

Geração de testes via LLM — abordagens

4 formas principais:

- 1. User story → teste E2E (mais comum)**
- 2. OpenAPI spec → contract test**
- 3. Recording de navegação → flow reproduzível**
- 4. Código de produção → unit tests**

Cada uma tem armadilhas distintas.

API de LLM — gerando teste Playwright

```
import Anthropic from '@anthropic-ai/sdk';

const client = new Anthropic();

const userStory = `
Como usuário autenticado, quero adicionar produto ao carrinho
e ver toast de "Adicionado com sucesso".
`;

const response = await client.messages.create({
  model: 'claude-opus-4-8',
  max_tokens: 2048,
  messages: [{
    role: 'user',
    content: `Gere teste Playwright em TypeScript para:\n${userStory}
Use locators por role/text/test-id.
Use Page Object Model.
Inclua assertions semânticas.`
  }],
});

console.log(response.content[0].text);
```

Exemplo com Claude (o que eu uso na demonstração). **Na prática do lab você usa**

Few-shot prompting — melhorar qualidade

Você é especialista em Playwright. Gere teste para a user story.

EXEMPLO:

US: "Como user, quero fazer login com email/senha e ir para dashboard."

Teste:

```
test('login com credenciais válidas', async ({ page }) => {  
  await page.goto('/login');  
  await page.getByLabel('Email').fill('user@email.com');  
  await page.getByLabel('Senha').fill('secret');  
  await page.getByRole('button', { name: 'Entrar' }).click();  
  await expect(page).toHaveURL('/dashboard');  
  await expect(page.getByText('Bem-vindo')).toBeVisible();  
});
```

USER STORY ATUAL:

[sua user story aqui]

Teste:

Few-shot ensina formato e nível de detalhe — resultado muito melhor que prompt simples.

Oracle problem — o risco fundamental

Teste gerado pode ser sintaticamente correto, executar sem erro, e **verificar a coisa errada**.

```
// Teste gerado por LLM — parece ok
test('checkout completa', async ({ page }) => {
  await page.getByRole('button', { name: 'Comprar' }).click();
  await expect(page.getByText('Sucesso')).toBeVisible();
});
```

Mas: o app deveria mostrar:

- Valor cobrado
- Número do pedido
- E-mail de confirmação enviado
- Atualização do estoque

Passou. Não testou o que importa.

Mitigando o oracle problem

1. **Cross-reference com spec** (OpenAPI, Gherkin): LLM recebe spec + US → asserções mais ricas
2. **Revisão humana obrigatória** em asserções — especialmente em fluxos de negócio crítico
3. **Mutation testing** complementar: prova que as asserções *detectam mudanças*
4. **Property-based testing**: gera casos, espera invariantes (sempre positivo, nunca duplicado)

IA escreve. Humano valida o oracle.

IA aplicada à análise Lighthouse

Em vez de só ler números, **alimentar relatório pra LLM**:

```
Você é especialista em performance web. Analise este Lighthouse report:
```

```
[JSON do report]
```

```
Identifique:
```

1. Quais regressões são significativas (semanticamente)?
2. Quais são triviais (variação de medição)?
3. Quais ações concretas recomendaria?

```
Responda em formato estruturado.
```

IA faz **interpretação semântica**, não só comparação numérica.

Quando IA brilha vs quando falha

Brilha:

- Transformação (story → estrutura de teste, imperativo → POM)
- Dados sintéticos realistas (nomes, CPFs, endereços)
- Análise interpretativa de relatórios (Lighthouse, coverage)
- Geração de variações de caso de teste

Falha:

- Definir o *que* assertar (oracle problem)
- Edge cases sutis de domínio específico
- Decisão arquitetural de cobertura
- Casos onde "passou" não significa "correto"

Pitfalls comuns

- ✗ **Aceitar testes IA sem revisar assertions** — oracle problem destrói confiança
- ✗ **Visual AI tolerância demais** — bug visual real escapa
- ✗ **Pixel diff sem anti-aliasing tolerance** — falsos positivos intermináveis
- ✗ **Custo/limite descontrolado** — LLM em loop sem teto de tentativas = \$\$ (ou estoura o free tier)
- ✗ **Few-shot com exemplos ruins** — garbage in, garbage out

Resumo

- **Pixel diff** → simples, muitos falsos positivos; **ML-based** (Applitools) → padrão em times sérios
- **Applitools**: modos Strict/Content/Layout + regiões ignoradas para dados dinâmicos
- **API de LLM** → geração de teste a partir de user story, OpenAPI, código
- **Few-shot prompting** → qualidade muito superior ao prompt simples
- **Oracle problem**: IA gera estrutura, humano valida o *que* assertar

Parte 3 — Prática guiada

Pipeline de geração + visual diff (lab IA)

A prática — pipeline no CineFav Web

```
cd exercicios/03-lab-ia-test-gen/pratica
ls src      # confirma: 01-gen-test.ts 02-healing-loop.ts 03-visual-diff.ts

npm install
cp .env.example .env      # seu token GitHub (GRÁTIS – instruções no arquivo)
```

- **Alvo dos testes gerados:** o CineFav Web (lab 02, já buildado)
- **Custo pro aluno: ZERO** — GitHub Models com o token da conta que você já tem (free tier = limite diário de requisições; o lab usa poucas)
- O pipeline **imprime os tokens** de cada chamada — colete pro relatório

Prática 1 — gerar teste de user story (01-gen-test.ts)

```
npm run gen -- stories/favoritar.story.md
```

```
1) Gerando teste a partir de stories/favoritar.story.md...  
   [tokens] in=812 out=534 (openai/gpt-4o-mini)  
2) Spec gravado em generated/favoritar.spec.ts  
3) Executando o teste gerado...  
✓ Teste gerado PASSOU de primeira.
```

- **Resolvido** — story com critérios + testids → LLM → spec → run
- Leia `prompts/gen-test.md` : as REGRAS do prompt são o que segura a qualidade
- **Oracle problem ao vivo:** o teste passou... verificando a coisa certa? Abra e leia.

Modelo grátis vs Claude — o que muda (demonstração)

No screencast eu rodo a **mesma story** nos dois:

	GitHub Models (grátis — o seu)	Claude (pago — demo do prof)
Teste sintaticamente ok	✓	✓
Segue as regras do prompt	✓ com prompt bem amarrado	✓ com folga
Riqueza das asserções	básicas (o que a story pediu)	infere verificações extras
Story vaga/longa	degrada rápido	segura melhor

- **Moral:** prompt restritivo + story curta compensam boa parte da diferença
- **Você não paga nada** — a comparação é demonstração, não é cobrada

Prática 2 — visual diff com pixelmach (03-visual-diff.ts)

```
npm run visual-diff -- screenshots/baseline.png screenshots/atual.png
```

- **Você completa os TODOs:** validar dimensões → pixelmach → drift % → gate 1%
- Experimente `threshold: 0` e veja o **anti-aliasing** acender falso positivo — é o slide dos 3 paradigmas acontecendo na sua máquina
- Baselines: gere com os snapshots do lab 02 (`test:visual:update`)

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Sua story** — escreva uma nova em `stories/` (fluxo de busca? detalhe?) e gere
- **Anote custo/latência** — vão direto pro relatório crítico do lab

Bons estudos!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith